Furuno DRS4DL+ Radar

Wire Protocol & System Technical Manual

J.C. Vaught
jvaught@sc.edu
University of South Carolina

March 2026

Contents

1 Introduction **2**
1.1 Hardware Overview . . . 2
1.2 Software Components . . . 2

2 Network Architecture **2**
2.1 Topology . . . 2
2.2 Port Map . . . 2
2.3 Auto-Discovery and Network Configuration . . . 2

3 Control Protocol **3**
3.1 Message Format . . . 3
3.2 Binary Keepalive . . . 4
3.3 Initialization Sequence . . . 4
3.3.1 Phase 1: Discovery . . . 4
3.3.2 Phase 2: Authentication . . . 5
3.3.3 Phase 3: Control Channel . . . 5
3.3.4 Phase 4: Firmware Query . . . 5
3.3.5 Phase 5: Register Dump . . . 5
3.3.6 Phase 6: Steady State . . . 5

4 Command Reference **5**
4.1 TX State (`$S69`) . . . 6
4.2 Range (`$S62`) . . . 6
4.3 Gain (`$S63`) . . . 6
4.4 Sea Clutter (`$S85`/`$S86`) . . . 7
4.5 Rain Clutter (`$S87`) . . . 7
4.6 Pulse Width (`$S65`) . . . 7
4.7 STC — Sensitivity Time Control (`$S64`) . . . 7

4.8 Main Bang Suppression (`$S83`) . . . 7
4.9 Bearing Resolution / Rez Boost (`$SEE`) . . . 7
4.10 Interference Rejection (`$S88`) . . . 8
4.11 Bird Mode (`$SBD`) . . . 8
4.12 Antenna Revolution Speed (`$S89`) . . . 8
4.13 Doppler / Echo Color Type (`$SEF`) . . . 8
4.14 Antenna Height (`$S82`) . . . 8
4.15 No-Transmit Sectors (`$S67`) . . . 8
4.16 Noise Reduction (`$S8A`) . . . 8
4.17 Scan Speed (`$S8C`) . . . 9

5 Periodic Telemetry **9**

6 Echo Data Format **9**
6.1 Packet Structure . . . 9
6.2 Echo Metadata . . . 10
6.3 Azimuth and Heading . . . 10
6.4 Echo Encoding Types . . . 10
6.4.1 Encoding 0: Raw Uncompressed . . . 10
6.4.2 Encoding 1: RLE with Repeat Marker . . . 10
6.4.3 Encoding 2: Delta RLE with Previous Spoke Reference . . . 10
6.4.4 Encoding 3: 2-Bit RLE with Dual Reference . . . 10

7 Doppler Processing **11**
7.1 Status Field Encoding . . . 11
7.2 Echo Byte Encoding . . . 11

8 Dual-Pulse Interleaving **12**
8.1 Pulse Types . . . 12
8.2 Range Dependence . . . 12
8.3 Rez Boost Effect . . . 12

9 PPI Rendering **13**
9.1 Coordinate Transformation . . . 13
9.2 Color Mapping . . . 13

10 System Architecture **13**
10.1 Module Layout . . . 13
10.2 Connection State Machine . . . 14

11 Web API Reference **14**
11.1 Power and Connection . . . 14
11.2 Radar Settings . . . 15
11.3 State Queries . . . 15
11.4 Echo Image . . . 15
11.5 Recording . . . 16
11.6 Pattern Execution . . . 16

12 CLI Modes **16**

13 Testing and Validation **17**
13.1 Test Categories . . . 17
13.1.1 Connection Tests (6) . . . 17
13.1.2 TX Tests (3) . . . 17
13.1.3 Settings Tests (14) . . . 17
13.1.4 Echo Impact Tests (2) . . . 17
13.1.5 Render Integration Test (1) . . . 18

14 Calibration System **18**
14.1 Pattern Step Definition . . . 18
14.2 Differential Command Generation . . . 18
14.3 Pattern Generation . . . 18

15 Proprietary Library Reference **18**

Introduction

This document provides a comprehensive technical reference for the Furuno DRS4DL+ marine radar system as interfaced through the `radar-client` Rust application. It covers the complete wire protocol, echo data encoding, system architecture, web API, and calibration procedures. All information was obtained through reverse engineering of the proprietary `libNAVNETDLL.so` shared library and live packet captures from the radar hardware.

The Furuno DRS4DL+ is a solid-state Doppler radar operating in the X-band (9.41 GHz). It communicates with its host processor over Ethernet using a combination of TCP control channels and UDP multicast for echo data delivery.

Hardware Overview

The radar unit (scanner) contains the antenna, transmitter, and receiver in a single radome enclosure. It connects to the host processor (in this case, a Raspberry Pi) via a standard Ethernet cable. The scanner has a fixed IP address of `172.31.3.226` on a `/16` subnet.

Software Components

The system consists of three main software components.

The **radar-client** is a Rust binary that implements the full Furuno wire protocol. It handles authentication, control commands, echo data reception and decoding, PPI rendering, and serves a web interface for browser-based operation.

The **web interface** is a set of static HTML/CSS/JavaScript files served by the Rust application. It provides real-time PPI display, radar settings controls, recording capabilities, and pattern-based calibration tools.

The **protocol-notes** directory contains the original reverse-engineering artifacts: packet captures, symbol tables from the proprietary library, and analysis documents that informed this implementation.

Network Architecture

Topology

The radar and host communicate over a dedicated Ethernet link on the `172.31.0.0/16` subnet. The host may also have additional network interfaces (Wi-Fi, Tailscale VPN) for external access to the web interface.

Port Map

Table 1 lists all network ports used by the system.

Auto-Discovery and Network Configuration

The radar broadcasts a UDP beacon on port 10010 approximately every 16 seconds. The `radar-client` application uses this beacon for auto-discovery. On startup, the client broadcasts probe packets on all detected network interfaces and listens for radar responses.

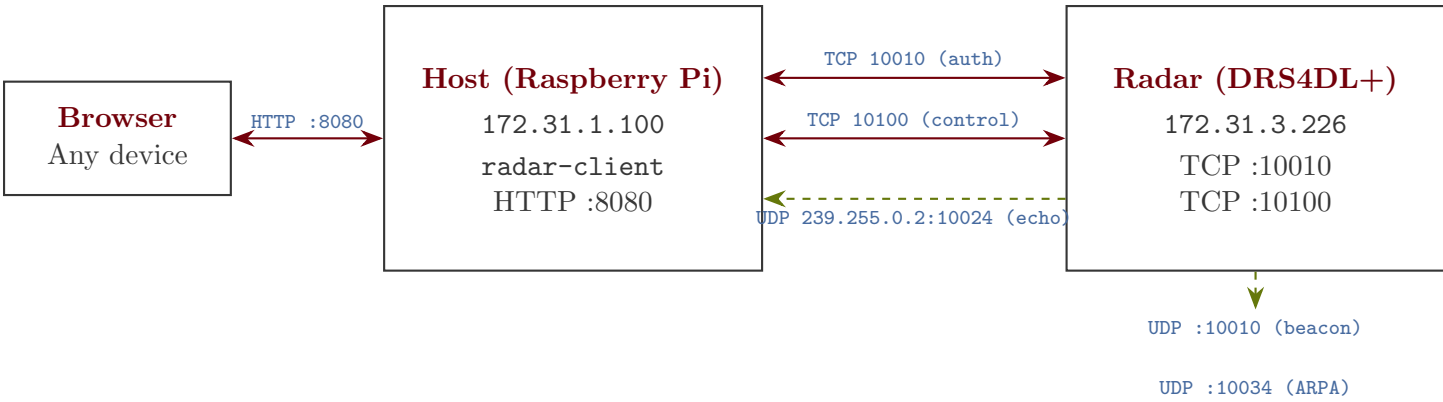


Figure 1: Network topology of the radar system.

Table 1: Network port assignments.

Port	Direction	Protocol	Purpose	Rate
10010	Host ↔ Radar	TCP	One-shot authentication	Once
10010	Radar → Broadcast	UDP	Discovery beacon	∼1/16 s
10024	Radar → 239.255.0.2	UDP	Echo sweep data	∼80/s
10034	Radar → Broadcast	UDP	ARPA target data	∼1/s
10100	Host ↔ Radar	TCP	Persistent control channel	∼2/s
8080	Browser ↔ Host	HTTP	Web interface	On demand

If the host has no interface on the radar's subnet, the application automatically adds a secondary IP alias to the physical Ethernet interface. This is handled in a platform-specific manner: `ip addr add` on Linux, `ifconfig alias` on macOS, and `netsh interface ip add address` on Windows.

The discovery process follows three stages. First, the client checks for explicit `--radar-ip` and `--host-ip` command-line flags. Second, if no flags are provided, it broadcasts UDP beacons on all interfaces for 3 seconds. Third, if discovery fails, it falls back to the default addresses (`172.31.3.226` and `172.31.1.100`) and ensures subnet reachability via auto-alias.

Control Protocol

The radar uses an NMEA-like ASCII protocol over TCP port 10100 for all control and status communication. A separate one-shot TCP connection on port 10010 handles initial authentication.

Message Format

All control messages follow the general format:

```
$<prefix><hex_id>,<param1>,<param2>,...\r\n
```

Three message prefixes are defined in Table 2.

Command IDs are two-character hexadecimal values (e.g., `62` for range, `69` for TX state). Parameters are comma-separated decimal integers. Messages are terminated with `\r\n` (0x0D 0x0A).

Table 2: Control message prefixes.

Prefix	Direction	Meaning
`$S`	Host → Radar	Set/Send command
`$R`	Host → Radar	Read/Request register
`$N`	Radar → Host	Notify/Acknowledge response

Binary Keepalive

In addition to ASCII messages, the control channel uses an 8-byte binary keepalive packet sent every 5–10 seconds.

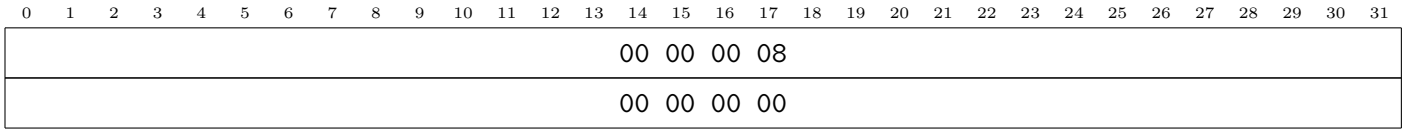


Figure 2: Binary keepalive packet (8 bytes). The first 4 bytes encode the packet length.

The keepalive must be distinguished from ASCII messages by checking the first byte: if byte 0 is `0x00` and the total length is 8, the packet is a keepalive rather than an ASCII command.

Initialization Sequence

The full cold-start initialization proceeds through six phases, illustrated in Figure 3.

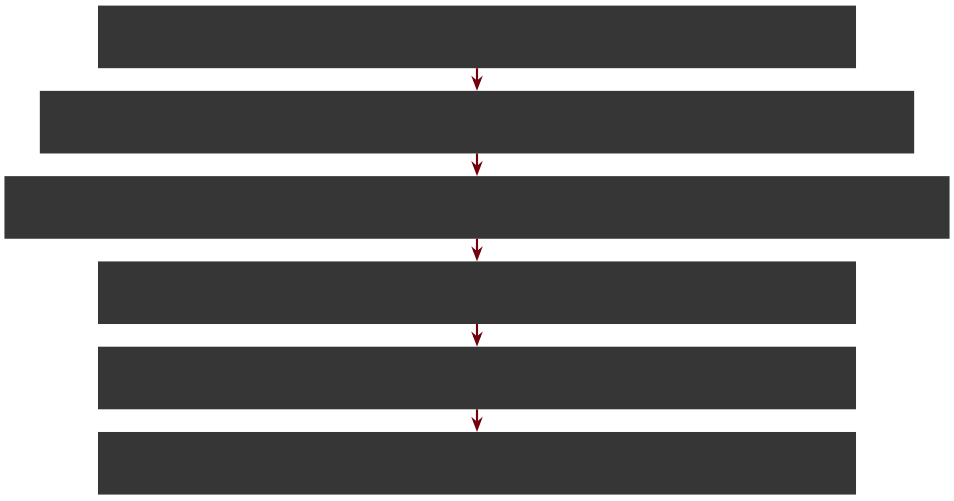


Figure 3: Initialization sequence phases.

Phase 1: Discovery

The radar continuously broadcasts a 32-byte discovery beacon on UDP port 10010 to the subnet broadcast address (`172.31.255.255`). The beacon contains the device identifier (e.g., “RD003226”) at byte offset 16.

```
01 00 00 01 00 00 00 00   (header)
```

```
00 01 00 18 01 00 00 00   (capability/type)
52 44 30 30 33 32 32 36   ("RD003226")
01 01 00 02 00 01 00 12   (device type/version)
```

Phase 2: Authentication

The host opens a TCP connection to the radar on port 10010 and sends a 56-byte binary payload containing a Furuno copyright string. The radar responds with a 12-byte acknowledgment.

```
Host sends (56 bytes):
  08 01 00 38 01 00 00 00 00 01 00 00
  COPYRIGHT (C) 2001 FURUNO ELECTRIC CO.,LTD.

Radar responds (12 bytes):
  09 01 00 0C 01 00 00 00 00 64 00 00
```

Authentication succeeds when byte 9 of the response equals `0x64` (decimal 100). The TCP connection is closed immediately after this exchange.

Phase 3: Control Channel

The host opens a persistent TCP connection to port 10100 with `TCP_NODELAY` enabled. The radar immediately sends a connection confirmation.

```
$N00,CLIENTCONNECTION,\r\n
```

Phase 4: Firmware Query

The host sends a firmware version request.

```
$R96,,,,,,,\r\n
```

The radar responds with firmware version strings for each subsystem.

```
$N96,0359360-01.07,0359358-01.01,0359359-01.02,0359361-01.06,,,
```

Phase 5: Register Dump

The host sends approximately 184 register read commands (`$R`) in rapid succession, covering register IDs from `0x61` through `0xFD`. The radar responds to each with a `$N` notification containing the current register values. This establishes the initial state of all radar parameters.

Phase 6: Steady State

After initialization, the control channel enters steady state. The radar periodically sends antenna heading reports (`$NAF,256`) at approximately 1 Hz and binary keepalive packets every 5–10 seconds. No echo data is transmitted until TX is explicitly enabled.

Command Reference

This section documents all implemented control commands. Each command is sent as an ASCII `$S` message on the TCP 10100 control channel.

TX State ($S69)

Controls the radar transmitter power state.

```
$S69,<state>,0,0,0,0,0\r\n
```

The `state` parameter is 1 for TX OFF (standby) and 2 for TX ON. Note the counterintuitive assignment where 2 enables transmission.

A TX toggle is always accompanied by three auxiliary commands sent atomically within approximately 1 ms.

```
$S69,2,0,0,0,0,0    (TX ON)
$S85,0               (A/C SEA reset)
$S86,0               (A/C SEA mode reset)
$S87,0               (A/C RAIN reset)
```

Range ($S62)

Sets the radar display range.

```
$S62,<wireIndex>,0,0,0\r\n
```

Table 3 maps wire indices to physical distances.

Table 3: Range wire index mapping.

Wire Index	Meters	Label
21	116	1/16 NM
0	231	1/8 NM
1	463	1/4 NM
2	926	1/2 NM
3	1,389	3/4 NM
4	1,852	1 NM
5	2,778	1.5 NM
6	3,704	2 NM
7	5,556	3 NM
8	7,408	4 NM
9	11,112	6 NM
10	14,816	8 NM
11	22,224	12 NM
12	29,632	16 NM
13	44,448	24 NM
14	59,264	32 NM
19	66,672	36 NM
15	88,896	48 NM

Gain ($S63)

Controls radar receiver gain.

```
$S63,<channel>,<value>,0,<maxGain>,0\r\n
```

The `channel` parameter selects between manual gain (0) and auto gain (1). For manual mode, the `value` parameter ranges from 0 to 255.

Sea Clutter ($S85/$S86)

Controls the A/C SEA (anti-clutter sea) filter.

```
$S85,<value>\r\n     (A/C SEA level, 0-255)
$S86,<value>\r\n     (A/C SEA mode)
```

Rain Clutter ($S87)

Controls the A/C RAIN (anti-clutter rain) filter.

```
$S87,<value>\r\n     (A/C RAIN level, 0-255)
```

Pulse Width ($S65)

Sets the transmitter pulse width.

```
$S65,<channel>,<value>,0,0,0,0,0\r\n
```

The `value` parameter ranges from 0 to 3, where 0 is the shortest pulse and 3 is the longest.

STC — Sensitivity Time Control ($S64)

Controls the near, middle, and far STC curves. The `channel` parameter selects the STC region.

```
$S64,<channel>,<value>,0,0,0,0\r\n
```

There are three STC regions: near (`channel`=0), middle (`channel`=1), and far (`channel`=2). Dedicated convenience commands also exist for individual STC regions.

```
$S85,<value>\r\n     (near STC)
$S86,<value>\r\n     (middle STC)
$S87,<value>\r\n     (far STC)
```

Main Bang Suppression ($S83)

Suppresses the main bang (transmitter pulse leakage) at close range.

```
$S83,<value>,0\r\n
```

Bearing Resolution / Rez Boost ($SEE)

Controls the bearing resolution enhancement.

```
$SEE,<level>,0\r\n
```

The `level` parameter takes values 0 (off), 1 (low), 2 (medium), or 3 (high). Rez boost affects the ratio of short to long pulse types in the dual-pulse echo data (see Section 8).

Interference Rejection ($S88)

Controls the interference rejection filter.

```
$S88,<enabled>\r\n
```

The `enabled` parameter is 0 for off and 1 for on.

Bird Mode ($SBD)

Controls bird echo suppression.

```
$SBD,<level>\r\n
```

The `level` parameter ranges from 0 (off) to 3 (maximum suppression).

Antenna Revolution Speed ($S89)

Controls the antenna rotation speed.

```
$S89,<speed>,<hiRpm>\r\n
```

The `speed` parameter selects the rotation mode. The `hiRpm` flag enables high-speed operation.

Doppler / Echo Color Type ($SEF)

Controls the Doppler target analyzer mode.

```
$SEF,<mode>,<sw>,<autoPicture>,0\r\n
```

The `mode` parameter takes values 0 (off), 1 (Target Analyzer), or 2 (alternative mode). The `sw` parameter enables (1) or disables (0) the feature. See Section 7 for details on how Doppler information is encoded in echo data.

Antenna Height ($S82)

Sets the antenna installation height above sea level.

```
$S82,<meters>,0,0,0,0\r\n
```

No-Transmit Sectors ($S67)

Defines angular sectors where the transmitter is inhibited.

```
$S67,<startAngle>,<endAngle>,<sectorIndex>,0\r\n
```

Up to 32 no-transmit sectors can be defined (sector indices 0–31). Angles are specified in tenths of a degree (0–3599).

Noise Reduction ($S8A)

Controls the echo noise reduction filter.

```
$S8A,<enabled>\r\n
```

Scan Speed (`$S8C`)

Controls the scan processing speed.

```
$S8C,<mode>\r\n
```

Periodic Telemetry

The radar sends unsolicited `$N` notifications on the control channel at regular intervals. Table 4 lists the observed periodic messages.

Table 4: Periodic telemetry messages.

Message	Rate	Content
`$NAF,256`	∼1/s	Antenna heading/rotation indicator
`$NF5,...`	∼1/s	Radar spoke/scan telemetry
`$NE0,...`	Periodic	Extended status (8 fields)
`$N83,100,1`	Periodic	Gain/threshold report
`$N68,...`	Periodic	Status with negative values
`$N00,CLIENTCONNECTION`	Once	Connection established notification
`[keepalive]`	5–10 s	8-byte binary keepalive

Echo Data Format

Echo data arrives via UDP multicast on address `239.255.0.2`, port 10024. Packets arrive at approximately 80 per second (∼12 ms interval) and contain one or more radar spokes (radial sweep lines).

Packet Structure

Each UDP echo packet consists of a 16-byte header followed by one or more spoke payloads.

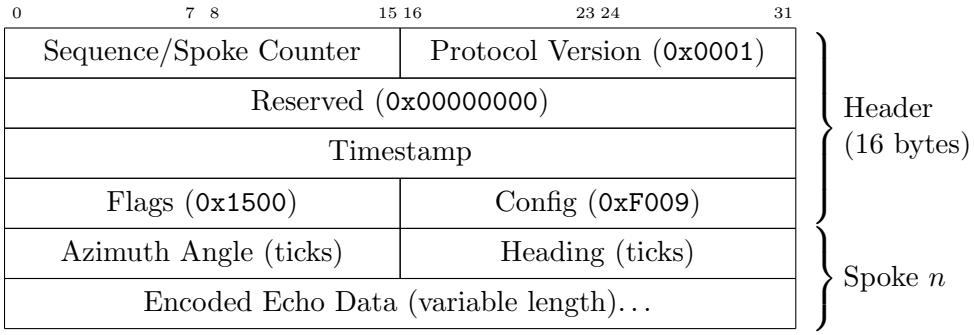


Figure 4: Echo UDP packet layout.

Echo Metadata

Key metadata is extracted from the 16-byte header. The sweep count (number of spokes in the packet) is derived from byte 9, bit-shifted right by 1. The sweep length (number of range bins per spoke) is encoded across bytes 10–11. The encoding type occupies bits 4:3 of byte 11.

$$\texttt{sweep_count} = \texttt{data[9]} \gg 1 \tag{1}$$

$$\texttt{sweep_len} = ((\texttt{data[11]} \,\&\, \texttt{0x07}) \ll 8) \mid \texttt{data[10]} \tag{2}$$

$$\texttt{encoding} = (\texttt{data[11]} \,\&\, \texttt{0x18}) \gg 3 \tag{3}$$

Azimuth and Heading

Each spoke begins with a 4-byte sub-header containing the azimuth angle and heading in 16-bit little-endian tick values. The radar uses 8,192 ticks per full rotation (360).

$$\theta_{\text{deg}} = \frac{\texttt{angle_ticks}}{8192} \times 360 \tag{4}$$

The heading field tracks the antenna's compass bearing independently of the spoke azimuth.

Echo Encoding Types

The radar uses four encoding schemes for echo amplitude data, selected by the encoding field in the packet header. All encodings produce a fixed-length array of range bins (typically 868 bins). After decoding, wire values are right-shifted by 2 bits for display: $\texttt{display} = \texttt{wire} \gg 2$.

Encoding 0: Raw Uncompressed

Echo bytes are stored directly with no compression. The decoded spoke is simply the first `sweep_len` bytes of the payload, zero-padded if necessary. Consumed bytes are rounded up to a 4-byte boundary.

Encoding 1: RLE with Repeat Marker

Uses single-byte run-length encoding. The least significant bit (LSB) distinguishes literal values from repeat markers.

If $\texttt{LSB} = 0$, the byte is a literal echo amplitude value. If $\texttt{LSB} = 1$, the byte encodes a repeat count for the previous amplitude value: $\texttt{count} = \texttt{byte} \gg 1$, with a count of 0 meaning 128 repeats.

Encoding 2: Delta RLE with Previous Spoke Reference

Similar to Encoding 1, but repeat markers copy from the *previous spoke* at the corresponding range bin index rather than repeating the last literal value. The first spoke in a packet uses Encoding 1 since no previous spoke reference is available.

Encoding 3: 2-Bit RLE with Dual Reference

Uses the two least significant bits for control. If $\texttt{bits}[1{:}0] = 00$, the byte is a literal value. If $\texttt{bit}[0] = 1$, repeat the previous literal value with count $= \texttt{byte} \gg 2$ (0 means 64). If $\texttt{bits}[1{:}0] = 10$, copy from the previous spoke with count $= \texttt{byte} \gg 2$ (0 means 64).

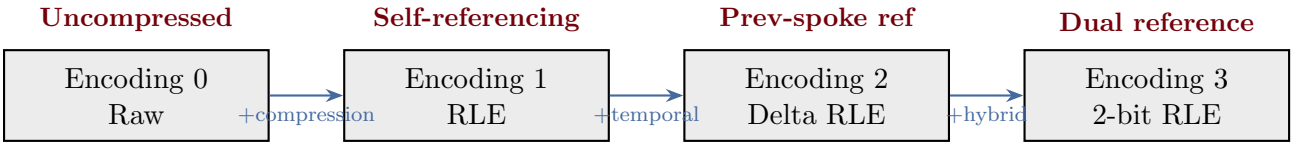


Figure 5: Progression of echo encoding complexity.

Doppler Processing

The DRS4DL+ supports Doppler target analysis, which repurposes the most significant bit of each echo byte to encode target motion information.

Status Field Encoding

The `status` field in each echo spoke carries the Doppler active flag in bit 8 (0x100).

Table 5: Status field bit layout.

Status	Binary	Meaning
1 (0x01)	`0_0000_0001`	Normal echo, long pulse (coarse)
4 (0x04)	`0_0000_0100`	Normal echo, short pulse (fine)
257 (0x101)	`1_0000_0001`	Doppler echo, long pulse
260 (0x104)	`1_0000_0100`	Doppler echo, short pulse

The confirmed bit layout is as follows. Bit 8 is the Doppler active flag (0 = normal, 1 = Doppler data present in echo bit 7). Bits 2:0 encode the pulse type (1 = long/coarse, 4 = short/fine, interleaved per spoke).

Echo Byte Encoding

When Doppler is **disabled** (status bit 8 = 0), the full 8-bit range is used for echo intensity (0–252). Approximately 26% of echo bytes have bit 7 set in normal operation.

When Doppler is **enabled** (status bit 8 = 1), the echo byte is split. Bit 7 serves as the Doppler direction flag (0 = stationary, 1 = moving target). Bits 6:0 encode echo intensity compressed to 7 bits (0–126).

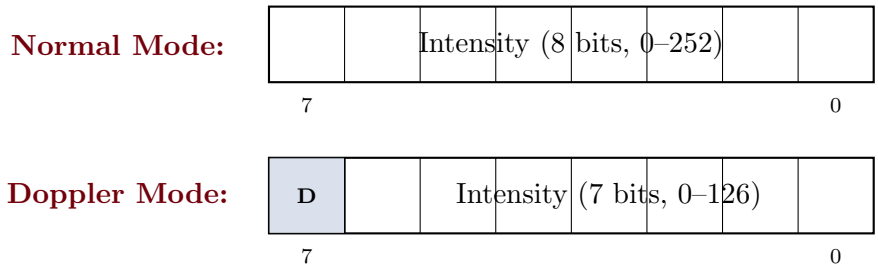


Figure 6: Echo byte layout in normal vs. Doppler mode. The D bit indicates a moving target.

Dual-Pulse Interleaving

The radar alternates between long and short transmitter pulses on consecutive spokes. This dual-pulse behavior is a hardware characteristic, not a software-controlled feature.

Pulse Types

Long-pulse spokes (status = 1) produce coarse quantization with only 17 discrete intensity levels.

```
0, 8, 20, 28, 36, 44, 56, 64, 76, 88, 104,
120, 140, 164, 188, 220, 252
```

Short-pulse spokes (status = 4) produce fine linear quantization with 130–240 unique intensity values, depending on rez boost settings.

Range Dependence

The ratio of short to long pulses varies with range setting.

Table 6: Dual-pulse behavior by range.

Range Index	Distance	Long %	Short %	Behavior
21	1/16 NM	50%	50%	Dual-pulse
0	1/8 NM	50%	50%	Dual-pulse
3	3/4 NM	50%	50%	Dual-pulse
6	2 NM	51%	49%	Dual-pulse
9	6 NM	69%	31%	Transitioning
12	16 NM	99.9%	0.1%	Single pulse only
15	48 NM	99.9%	0.1%	Single pulse only

At short ranges (<6 NM), both pulse types are interleaved approximately equally. At long ranges (>16 NM), only the long pulse is used.

Rez Boost Effect

The bearing resolution boost does not control whether dual-pulse occurs, but it shifts the ratio and increases the intensity resolution of short-pulse spokes.

Table 7: Effect of rez boost on dual-pulse ratio and resolution.

Rez Boost	Long %	Short %	Short Unique Values
Off (0)	81%	19%	∼130
Low (1)	37%	63%	∼210
Med (2)	39%	61%	∼240
High (3)	36%	64%	∼240

PPI Rendering

The Plan Position Indicator (PPI) display renders decoded echo spokes onto a circular image. The implementation produces an 800×800 pixel RGBA image.

Coordinate Transformation

Each spoke is drawn as a radial line from the image center outward. The radar uses compass bearing convention (0 = north, clockwise), which must be converted to mathematical angle convention (0 = east, counter-clockwise).

$$\theta_{\text{math}} = \frac{\pi}{2} - \theta_{\text{compass}} \tag{5}$$

For each range bin r along a spoke at angle θ, the pixel coordinates are computed as follows.

$$x = x_{\text{center}} + r \cdot \frac{R_{\text{max}}}{N_{\text{bins}}} \cdot \cos(\theta_{\text{math}}) \tag{6}$$

$$y = y_{\text{center}} - r \cdot \frac{R_{\text{max}}}{N_{\text{bins}}} \cdot \sin(\theta_{\text{math}}) \tag{7}$$

where $R_{\text{max}} = 398$ pixels (center to edge minus 2-pixel margin) and $N_{\text{bins}} = 868$ is the number of range bins per spoke.

Color Mapping

Echo intensity values (0–63 after the $\gg 2$ display conversion) are mapped to a green-channel intensity scale on a black background. Zero-amplitude bins remain black; non-zero bins are rendered with intensity proportional to their value.

System Architecture

Module Layout

The Rust application is organized into eight modules. Figure 7 illustrates the dependency relationships.

`main.rs` Entry point, CLI argument parsing, IP resolution, mode dispatch.

`protocol.rs`
: Wire protocol implementation: constants, command builders, echo decoders, discovery, network interface detection.

`connection.rs`
: `RadarConnection` state machine: auth, control channel, init, TX control, keepalive, response parsing. Also provides the `create_echo_socket` helper.

`echo.rs` Echo packet header parsing, rotation statistics collection, CSV export.

`render.rs`
: PPI image rendering (spoke-to-pixel transformation, PNG encoding).

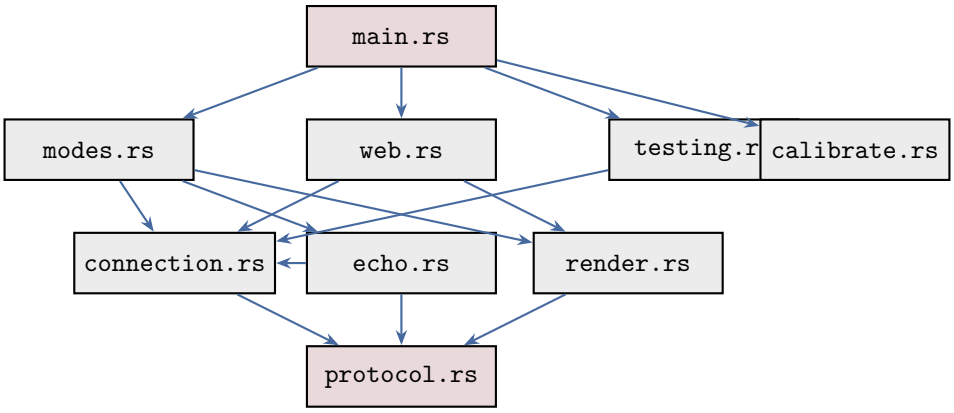


Figure 7: Module dependency graph. Shaded modules are the primary entry points.

`modes.rs` CLI mode implementations: handshake, TX toggle, echo listen, full cycle, dump raw, render live/CSV, discover.

`web.rs` HTTP server (Axum), all REST API endpoints, echo listener thread, pattern execution engine.

`testing.rs`
Validation suite, A/B command testing with before/after PPI captures.

`calibrate.rs`
Calibration pattern generator, multi-parameter sweep runner.

Connection State Machine

The `RadarConnection` struct manages the radar lifecycle through five states.



Figure 8: RadarConnection state machine.

The convenience method `connect_full()` chains `auth()` → `connect_control()` → `init()` in sequence.

Web API Reference

The web server runs on the host at port 8080 (configurable) and provides a REST API for browser-based radar control. All endpoints return JSON or raw image data.

Power and Connection

Endpoint	Method	Description
/toggle_echo	GET	Toggle TX ON/OFF. Returns "on" or "off".
/reconnect	GET	Reconnect to the radar (full init). Returns "ok" or error.

Radar Settings

All settings endpoints accept parameters via query string (e.g., /set_range?value=6).

Endpoint	Parameter	Description
/set_range	value (wire index)	Set radar range
/set_gain	value (0–255)	Set gain level
/set_pulse_width	value (0–3)	Set pulse width
/set_bearing_resolution	value (0–3)	Set rez boost level
/set_acsea	value (0–255)	Set sea clutter filter
/set_acrain	value (0–255)	Set rain clutter filter
/set_near_stc	value	Set near STC
/set_middle_stc	value	Set middle STC
/set_far_stc	value	Set far STC
/set_main_bang	value	Set main bang suppression
/set_interference_rejection	value (0–1)	Toggle IR filter
/set_bird_mode	value (0–3)	Set bird mode level
/set_doppler	value (0–2)	Set Doppler mode
/set_antenna_revolution	value	Set rotation speed

State Queries

Endpoint	Method	Description
/get_current	GET	Returns JSON with current radar settings (range, gain, pulse, etc.)
/get_system_info	GET	Returns JSON with firmware version, connection status, TX state
/system_metrics	GET	Returns JSON with system metrics (CPU, memory, uptime)

Echo Image

Endpoint	Method	Description
/newest_png	GET	Returns the latest PPI image as PNG (800×800 RGBA)
/newest_trail	GET	Alias for /newest_png

Endpoint	Method	Description
/newest_detection	GET	Alias for /newest_png

Recording

Endpoint	Method	Description
/toggle_recording	GET	Start/stop echo CSV recording. Creates timestamped output directory.

Pattern Execution

Pattern execution allows automated parameter sweeps for calibration and data collection.

Endpoint	Method	Description
/run_pattern	GET	Run a single pattern step (query params: range, gain, pulse, rezboost, speed, interp, repeat)
/run_pattern_queue	POST	Run a sequence of pattern steps. Body format: repeat:base64(steps) where steps are semicolon-separated
/stop_pattern	GET	Stop the currently running pattern
/pattern_status	GET	Returns running|detail or idle|detail

Each pattern step is encoded as a comma-separated string: range,gain,pulse,rezboost,speed,interp,repe
Steps are separated by semicolons. The entire step string is base64-encoded in the POST body.

CLI Modes

The radar-client binary supports multiple operating modes selected by the first positional argument.

```
radar-client [--radar-ip IP] [--host-ip IP] <mode> [args...]
```

Mode	Description
handshake	Perform full authentication and initialization. Print firmware info and register values.
tx-on	Initialize and enable transmitter.
tx-off	Initialize and disable transmitter.
echo	Listen for multicast echo packets without authentication (passive monitoring).
full	Full cycle: init, TX ON, collect one rotation of echo data, TX OFF.
command	Send test commands and display acknowledgments.

Mode	Description
`render-csv` *file* `[`*out*`]`	Render a previously recorded CSV file as a PPI image (offline, no radar needed).
`render-live [`*out*`]`	Capture one rotation and render to PNG.
`dump-raw [`*count*`]`	Dump raw echo packet headers and hex data.
`test-commands`	A/B command verification with before/after PPI captures.
`calibrate` *args*	Run calibration pattern sweeps with configurable parameter ranges.
`discover`	Broadcast discovery beacons and listen for radar responses.
`validate`	Run the comprehensive 6-phase validation suite.
`web [`*port*`] [`*webdir*`]`	Start the HTTP server (default port 8080, web files from `../web`).

Testing and Validation

The project includes 27 integration tests that require a live radar connection. Tests must be run sequentially since the radar only supports one active control connection.

```
cargo test -- --test-threads=1
```

Test Categories

Connection Tests (6)

These tests verify the fundamental communication path: TCP authentication, control channel establishment, full initialization, firmware version retrieval, keepalive exchange, and multicast echo socket creation.

TX Tests (3)

These tests verify transmitter control: TX ON/OFF cycling, echo packet reception after TX ON, and full rotation data collection (8,192 spokes).

Settings Tests (14)

Each test follows a set-and-read pattern: send a `$S` command to change a parameter, then send the corresponding `$R` query to verify the change was accepted. Covers range, manual gain, auto gain, pulse width, rez boost, sea clutter, rain clutter, near/middle/far STC, main bang suppression, interference rejection, bird mode, Doppler, and antenna revolution speed.

Echo Impact Tests (2)

These tests verify that parameter changes produce measurable effects in the echo data. One test compares echo amplitudes at different gain levels; the other verifies that range changes produce different data patterns. These tests require physical targets in the radar's field of view to produce reliable results.

Render Integration Test (1)

Captures a full rotation of live echo data and renders it to a PNG file, verifying the complete pipeline from UDP multicast reception through spoke decoding to PPI image generation.

Calibration System

The calibration system enables automated parameter sweeps for systematic data collection. A calibration pattern defines a sequence of radar configurations, each held for a specified number of antenna rotations.

Pattern Step Definition

Each step specifies seven parameters.

```
ControlStep {
    range:          u16,    // wire index (0-21)
    gain:           u16,    // 0-255
    pulse_length:   u16,    // 0-3
    rez_boost:      u16,    // 0-3
    rotation_speed: u16,    // antenna speed mode
    interpolation:  u16,    // spoke interpolation target
    repeat:         u16,    // rotations to hold this config
}
```

Differential Command Generation

When transitioning between pattern steps, only the parameters that changed are sent to the radar. This minimizes control channel traffic and avoids unnecessary state transitions.

Pattern Generation

The `generate_pattern` function creates a full combinatorial sweep across multiple parameter ranges, producing all possible combinations of the specified values for systematic characterization.

Proprietary Library Reference

The original Furuno `libNAVNETDLL.so` shared library was reverse-engineered to develop this implementation. Table 15 summarizes the library's module structure. The full symbol classification (6,102 defined functions) is available in `protocol-notes/symbols.md`.

Table 15: libNAVNETDLL.so module breakdown.

Module	Prefix	Count	Relevance
Radar Command Set	`RmcSet*`	154	Reimplemented
Radar Command Get	`RmcGet*`	143	Reimplemented
Radar Command Req	`RmcReq*`	133	Reimplemented
Radar Module	`Rm*`	67	Reimplemented
NAVNET Common	`Nc*`	310	Reimplemented
ARPA	`Rma*`	13	Not yet
Echo Sounder	`Esc*/Es*`	882	Not applicable
FNET Network	`nn*`	188	Internal
Licensing	`Flm*`	46	Bypassed
Crypto	`Blowfish*/bf*`	14	Not needed